

Capítulo 1

Resultados

El propósito de este capítulo es reconstruir, paso a paso, cómo se construye un detector de alucinaciones para sistemas RAG con aprendizaje automático clásico, y presentar los resultados que alcanza. El recorrido sigue el orden natural de un problema de aprendizaje: primero se presenta la base de datos y se caracteriza el fenómeno; después se preparan los datos mediante ingeniería de características y se selecciona un subconjunto de variables; a continuación se exponen las métricas y la metodología de evaluación; luego se entrena y compara cada modelo; y por último se sitúa el resultado frente al estado del arte. Dos aspectos reciben atención especial por su peso en este problema: la *ingeniería de características*, que es donde reside casi toda la señal, y el *desbalance de clases*, que condiciona la decisión final. La exploración de las vías que no funcionaron se recoge, de forma condensada, al final del capítulo.

1.1. Presentación de la base de datos

1.1.1. Introducción y problema

Un sistema de generación aumentada por recuperación (RAG) responde a una consulta en dos fases: recupera uno o varios documentos relevantes y, condicionado a ellos, genera una respuesta en lenguaje natural [13]. El sistema alucina cuando la respuesta afirma algo que el documento recuperado no respalda [11]. Por ejemplo, ante una fuente que dice “*completó el maratón de Boston, de 42 km*”, una respuesta que afirme “*lo completó en 26,2 horas*” es una alucinación: el número existe en el dominio, pero la afirmación no está sostenida por la fuente.

Detectar estas alucinaciones importa porque un sistema RAG que inventa datos con tono seguro es peligroso en aplicaciones reales. Hoy se detectan sobre todo con un modelo de lenguaje grande actuando como juez, lo que es caro y opaco. El objetivo de este trabajo es construir un detector *clásico*, barato e interpretable, que dado el par (respuesta, fuente) decida si la respuesta contiene alguna alucinación, usando solo características léxicas y estadísticas y un clasificador clásico. La hipótesis falsable que guía el trabajo es que tal detector alcanza un AUC-ROC de al menos 0,80 sobre RAGTruth con una evaluación honesta.

1.1.2. Composición y variables iniciales

Se emplea RAGTruth [14], un corpus de salidas de modelos de lenguaje anotadas por humanos según contengan o no alucinaciones. Cubre tres tareas: respuesta a preguntas

(QA), generación de texto a partir de datos estructurados (Data2txt) y resumen de documentos (Summary). Se trabaja con la partición oficial: 15 090 respuestas de entrenamiento y 2 700 de test. Cada registro aporta las variables de origen del Cuadro 1.1; nótese que la entrada del problema es *texto*, no un vector numérico, lo que motiva la ingeniería de características de la sección siguiente.

Variable	Descripción
response	Respuesta generada por el modelo de lenguaje (el texto a auditar).
source	Documento fuente recuperado (el contexto que debe respaldar la respuesta).
task_type	Tarea: QA, Data2txt o Summary.
model	Modelo de lenguaje que generó la respuesta.
hallucination_labels	Anotación humana: lista de tramos (<i>spans</i>) alucinados, con su tipo.

Tabla 1.1: Variables de origen de RAGTruth. La variable objetivo se deriva de **hallucination_labels** (sección 1.1.3).

1.1.3. De las anotaciones a la variable objetivo binaria

La anotación **hallucination_labels** marca, dentro de cada respuesta, los tramos concretos que alucinan. A partir de ella se define la variable objetivo binaria del problema: una respuesta se etiqueta **label** = 1 (alucina) si contiene al menos un tramo anotado, y **label** = 0 (limpia) en caso contrario. La anotación aparece en tres formatos en el corpus (lista vacía, lista con tramos, o cadena de texto que representa la lista), y los tres se normalizan al construir la etiqueta. La tarea es, por tanto, de clasificación binaria a nivel de respuesta.

1.1.4. Los cuatro tipos de alucinación

RAGTruth distingue cuatro tipos de alucinación, cruzando dos ejes: si el error es *evidente* o *sutil*, y si consiste en información *sin base* (inventada) o en un *conflicto* directo con la fuente. El Cuadro 1.2 muestra su composición por tarea. El dato relevante para lo que sigue es que Summary es en su gran mayoría de tipo evidente (aproximadamente el 86 %), y aun así, como veremos, es la tarea más difícil de detectar; el tipo anotado no explica la dificultad. El descriptivo de los tramos (Figura 1.1) completa el retrato: dominan los números, las fechas y los nombres propios, lo que justifica de forma natural las características de solape numérico y de entidades.

Tarea	Evidente sin base	Evidente conflicto	Sutil sin base	Sutil conflicto
Data2txt	0,383	0,447	0,163	0,007
QA	0,522	0,146	0,314	0,018
Summary	0,514	0,343	0,100	0,043

Tabla 1.2: Composición de los tipos de alucinación por tarea (fracción de tramos). Summary es mayoritariamente evidente y aun así la tarea difícil.

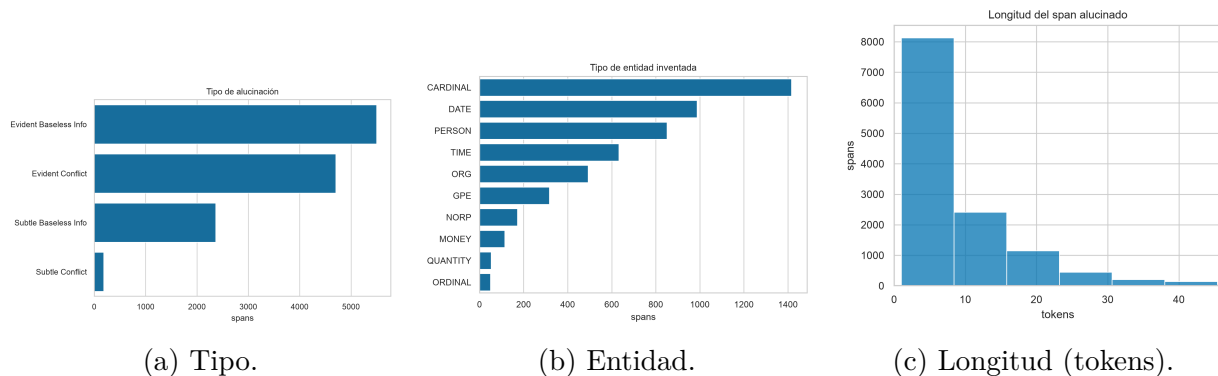


Figura 1.1: Descriptivo de los tramos alucinados. Predominan números y entidades nombradas.

1.1.5. El desbalance de clases

La prevalencia global de alucinación es del 44,5 % en entrenamiento y del 34,9 % en test. El desbalance global es moderado, pero engaña: al abrirlo por tarea (Cuadro 1.3 y Figura 1.2) aparece la estructura real. Data2txt es mayoritariamente positiva; QA y Summary son minoritariamente positivas, y su prevalencia cae todavía más en el test. Estamos ante un desbalance *heterogéneo por tarea* y con *desplazamiento* entre entrenamiento y test. La consecuencia práctica, un único umbral de decisión no puede ser óptimo para prevalencias tan dispares, se aborda en la metodología (sección 1.4.3).

Tarea	prevalencia (train)	prevalencia (test)
Data2txt	0,694	0,643
QA	0,311	0,178
Summary	0,311	0,227
Global	0,445	0,349

Tabla 1.3: Prevalencia de alucinación por tarea. El desbalance difiere entre tareas y se desplaza train→test de forma desigual.

1.2. Preparación de los datos: ingeniería de características

Como la entrada es texto, el primer paso es transformar cada par (respuesta, fuente) en un vector numérico de características que capturen indicios de alucinación. Esta sección es el núcleo del trabajo: define las características candidatas, comprueba si separan las clases, y selecciona un subconjunto mediante un método formal.

1.2.1. Tokenización y notación

Toda característica se construye sobre una tokenización simple: el texto se pasa a minúsculas y se extraen las secuencias alfanuméricas como tokens; para las características numéricas se usa una variante que conserva los decimales como una sola pieza (“26,2” es

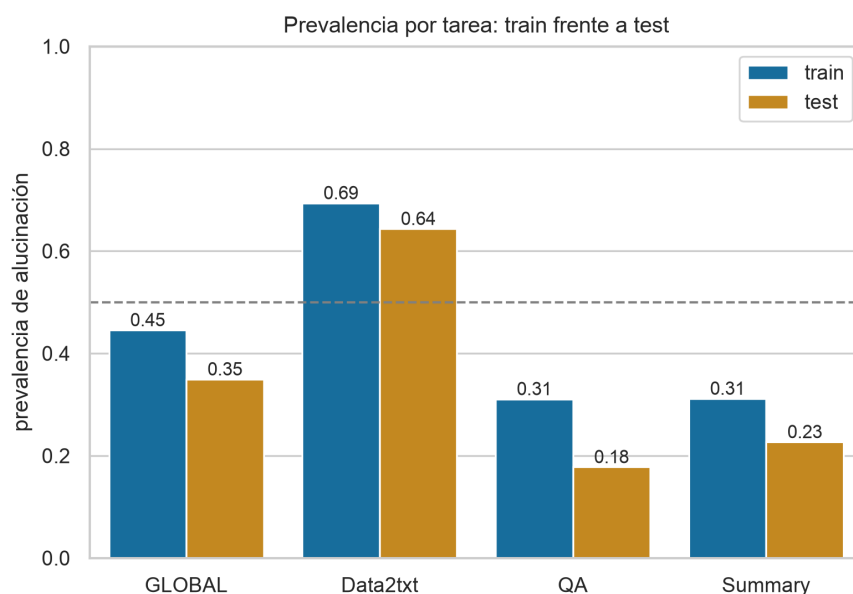


Figura 1.2: Prevalencia por tarea en entrenamiento y test. El desplazamiento es mayor en QA (0,311→0,178).

un token), necesaria para los n -gramas y la consistencia numérica. Sobre esta tokenización se definen los dos conjuntos básicos.

Definición 1. Dada una respuesta y su fuente, sea R el conjunto de palabras únicas de la respuesta y F el conjunto de palabras únicas de la fuente.

R y F son la base de casi todas las características: la idea recurrente es que una respuesta fiel reutiliza el vocabulario de la fuente, mientras que una alucinación introduce o recombina términos.

1.2.2. Catálogo de características candidatas

Se construyen dieciocho características candidatas, agrupadas en cuatro familias. El Cuadro 1.4 las recoge con su fórmula y la señal que persiguen. La familia de solape léxico mide cuánto vocabulario comparte la respuesta con la fuente; la numérica y relacional busca cifras y unidades fabricadas; la de nivel frase agrega el soporte frase a frase, con la intuición de que una alucinación suele ser *una* frase sin respaldo que el promedio global diluye, por lo que se agrega con el mínimo; y la codificación *one-hot* de la tarea permite al modelo especializar su decisión por tipo de tarea. El coseno TF-IDF [18] aporta una medida de similitud clásica.

1.2.3. ¿Separan las características las clases?

Antes de seleccionar, conviene ver si estas características discriminan. La Figura 1.3 muestra la distribución de cada característica continua separada por la variable objetivo. La señal es visible: en `containment`, `jaccard` o `num_context`, la clase alucinada se desplaza hacia menos solape. La Figura 1.4 muestra que muchas características están fuertemente correlacionadas entre sí (el bloque de solape léxico ronda correlaciones de 0,8 a 0,98), lo que anticipa que sobran variables.

Característica	Definición	Señal que captura
<code>containment</code>	$ R \cap F / R $	precisión léxica (señal dominante)
<code>jaccard</code>	$ R \cap F / R \cup F $	solape simétrico
<code>tfidf_cos</code>	coseno TF-IDF respuesta-fuente	similitud distribucional
<code>num_overlap</code>	fracción de números de R en F	cifras/fechas inventadas
<code>novel_bigram</code>	fracción de bigramas de R no en F	recombinación de pares
<code>novel_trigram</code>	ídem con trigramas	recombinación estricta
<code>num_context</code>	fracción de números con su unidad en F	cambio de unidad/atributo
<code>neg_diff</code>	diferencia de densidad de negación	conflicto de polaridad
<code>answer_len</code>	longitud de la respuesta (tokens)	longitud
<code>sent_cont_*</code>	containment por frase (mín/media/desv/% bajo)	frase peor sostenida
<code>sent_sim_*</code>	similitud por frase (mín/media)	frase menos parecida
<code>task_*</code>	one-hot de la tarea (QA/Summary/Data2txt)	especialización por tarea

Tabla 1.4: Las dieciocho características candidatas. R y F son los conjuntos de palabras únicas de respuesta y fuente.

1.2.4. Selección de variables con RFE y el método del codo

Para elegir un subconjunto se usa *Recursive Feature Elimination* (RFE) [10], un método envolvente: se entrena el modelo con todas las características, se elimina la menos importante según su importancia interna, se reentrena y se repite. El orden inverso de eliminación da un ranking de más a menos importante. A continuación se traza el rendimiento en validación cruzada (GroupKFold por `source`) frente al número de variables añadidas en ese orden, y se fija el número por el *método del codo* sobre la curva de F1: el punto a partir del cual añadir variables deja de aportar.

La Figura 1.5 muestra la curva. El AUC sube de golpe hasta unas seis variables y luego se aplana; el F1 alcanza su mejor valor en $k = 11$ y a partir de ahí no mejora. Se fija el conjunto en **once variables**. La característica `tfidf_cos` no entra en la selección (su aporte es de cola, por debajo de 0,004 de importancia), y se acepta el veredicto del método sin forzarla. Las tres variables *one-hot* de tarea sí entran.

1.2.5. El conjunto elegido

Las once variables definitivas, en orden de importancia, son:

`containment`, `task_QA`, `task_Summary`, `task_Data2txt`, `num_context`, `answer_len`, `jaccard`,
`sent_cont_min`, `novel_bigram`, `num_overlap`, `sent_sim_min`.

El conjunto es interpretable: `containment` domina, acompañada de la consistencia numérica (`num_context`, `num_overlap`), la recombinación (`novel_bigram`), el soporte por

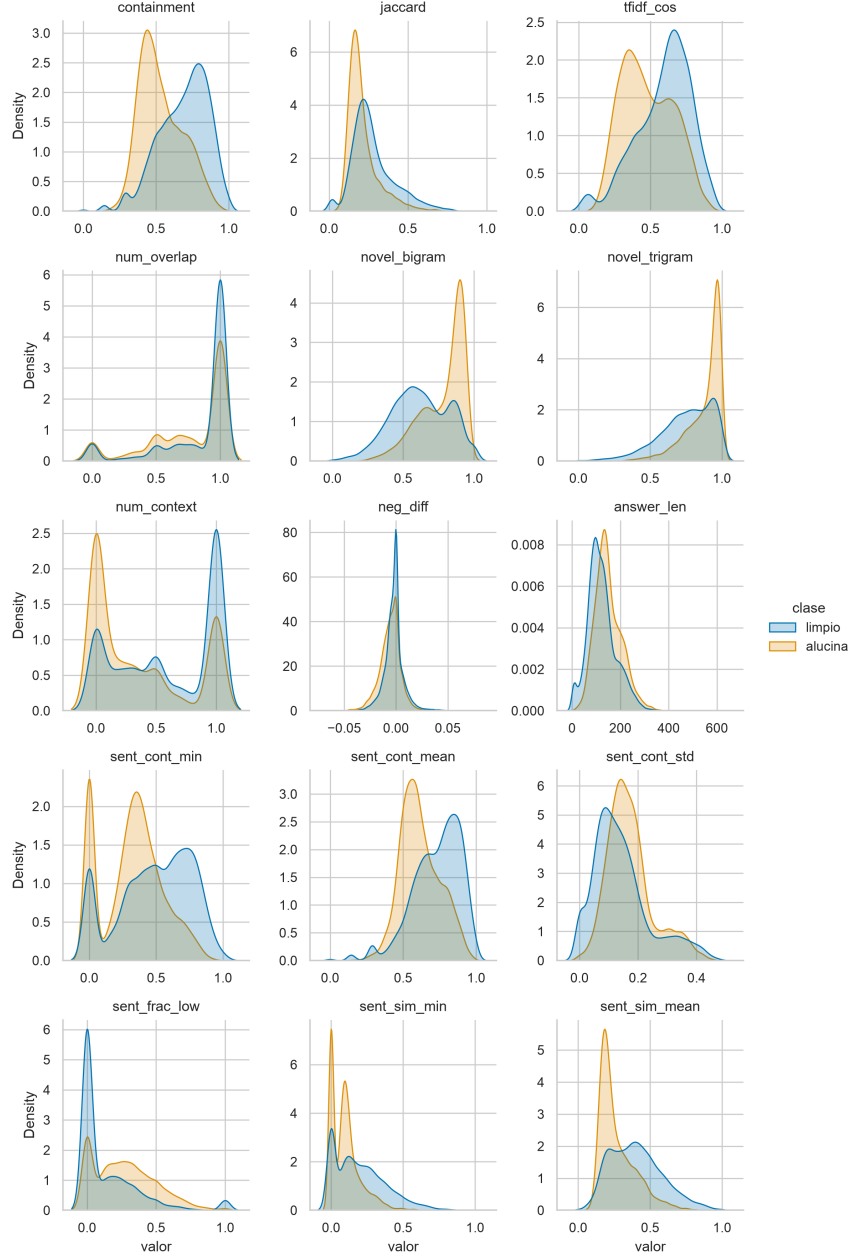


Figura 1.3: Distribución de cada característica continua separada por clase. `containment` y las medidas de solape muestran el mayor desplazamiento.

frase (`sent_cont_min`, `sent_sim_min`) y la longitud, más la codificación de tarea. Sobre este conjunto se hace todo el estudio posterior. Su separabilidad por clase y su correlación aparecen en la Figura 1.6.

1.3. Métricas

El propósito de esta sección es presentar las métricas con las que se evalúan los modelos. Todas se construyen sobre cuatro conteos básicos de la matriz de confusión (Figura 1.7), tomando como clase positiva la alucinación: verdaderos positivos (VP), verdaderos negativos (VN), falsos positivos (FP, error de tipo I) y falsos negativos (FN, error de tipo II). En detección de alucinaciones el FN es el error caro: una alucinación que se cuela sin detectar.

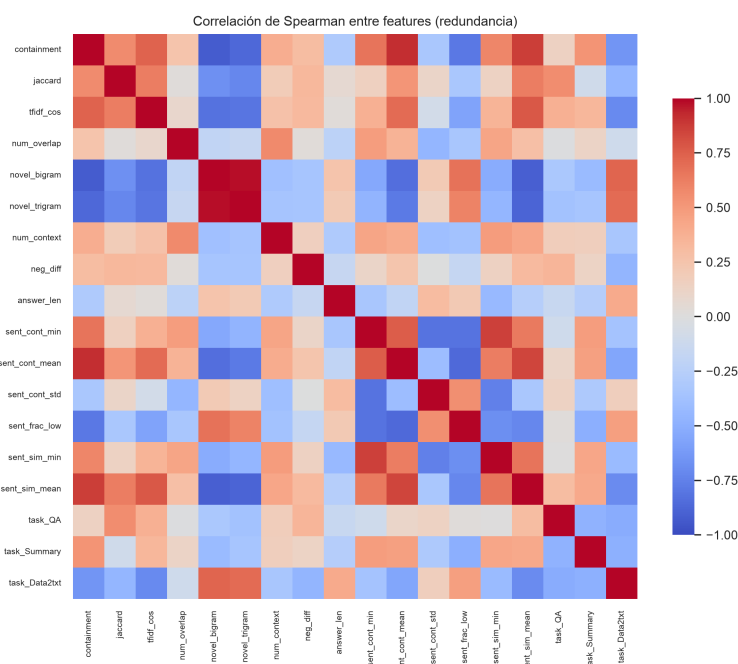


Figura 1.4: Correlación de Spearman entre las características candidatas. El bloque de solape léxico está muy correlacionado: hay pocas señales independientes.

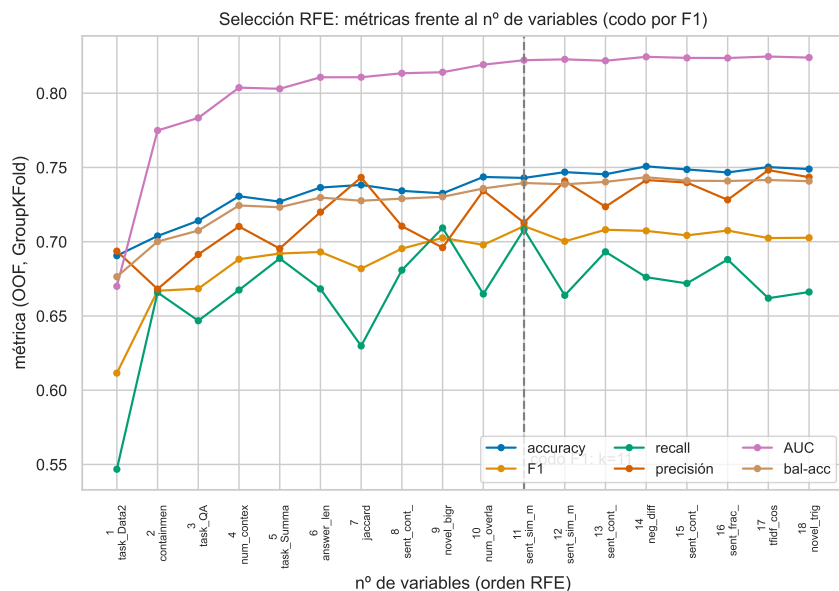
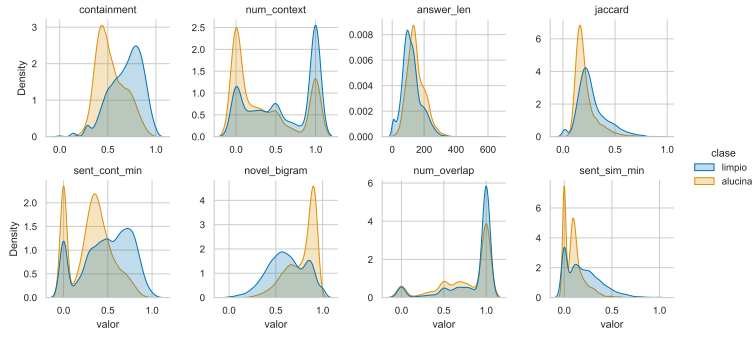
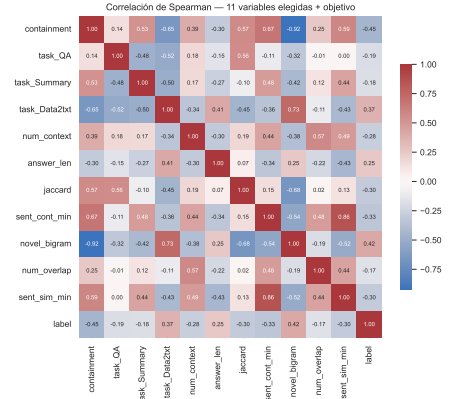


Figura 1.5: Selección RFE: las seis métricas (OOF, GroupKFold) frente al número de variables, en orden RFE. El codo sobre F1 fija $k = 11$; el rango 6–18 está prácticamente empatado en AUC.



(a) Separabilidad por clase.



(b) Correlación de Spearman.

Figura 1.6: Descriptivo del conjunto de once variables elegido.

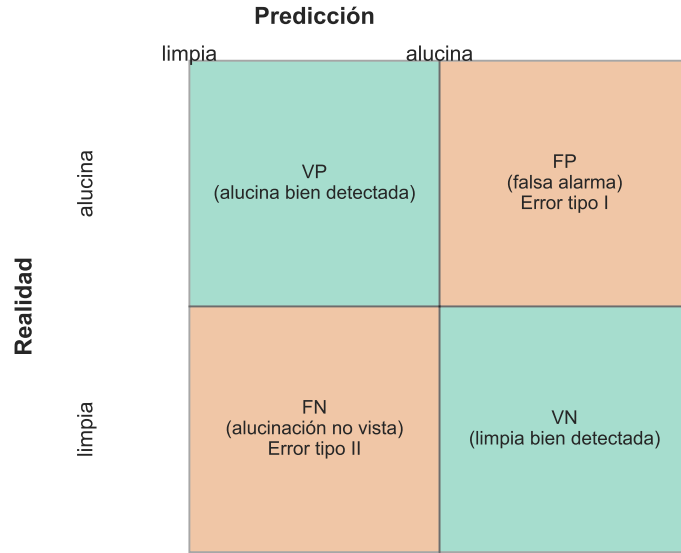


Figura 1.7: Matriz de confusión. La clase positiva es la alucinación; el falso negativo (tipo II) es una alucinación no detectada.

A partir de estos conteos se definen la precisión y la exhaustividad (recall):

$$\text{precisión} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN}. \quad (1.1)$$

La precisión mide cómo de acertado es el modelo cuando predice alucinación; el recall, cuántas de las alucinaciones reales caza. Ambas se mueven en sentidos opuestos según el umbral de decisión: exigir más para predecir positivo sube la precisión y baja el recall, y viceversa. La curva precisión-recall resume ese equilibrio.

La curva ROC contrapone la tasa de verdaderos positivos (el recall) frente a la tasa de falsos positivos al variar el umbral, y su área, el AUC-ROC [8], resume el rendimiento en todos los umbrales. El AUC tiene dos propiedades que lo hacen la métrica interna del trabajo: es invariante a la escala y *invariante al umbral*, es decir, mide la separación pura entre clases. El valor F1 combina precisión y recall en su media armónica, y su

generalización F_β pondera el recall β^2 veces más que la precisión:

$$F_1 = 2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}, \quad F_\beta = (1 + \beta^2) \frac{\text{precisión} \cdot \text{recall}}{\beta^2 \text{precisión} + \text{recall}}. \quad (1.2)$$

Se reporta el AUC como métrica interna (cumple la hipótesis $\geq 0,80$ y es estándar), y el F1 y la accuracy para comparar con el estado del arte, que se mide en esas dos.

1.4. Metodología de entrenamiento y validación

Esta sección describe el flujo desde la base de datos en crudo hasta la evaluación final, resumido en el esquema de la Figura 1.8, y los tres problemas que hay que resolver para que la evaluación sea honesta.

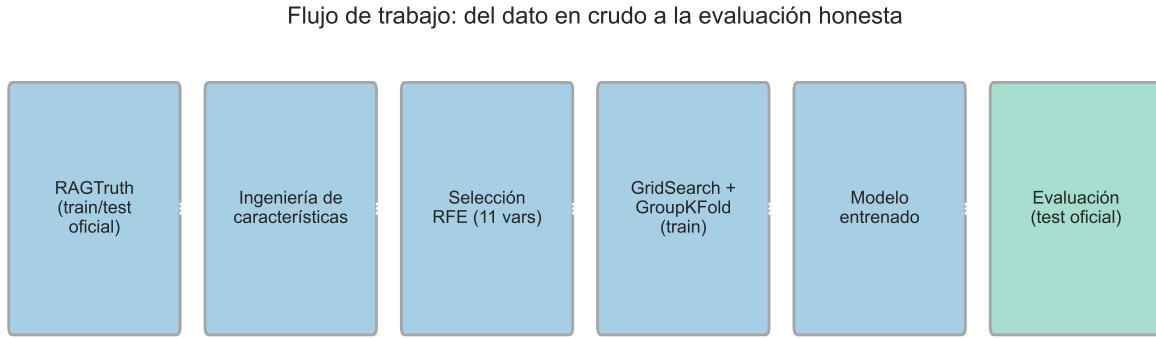


Figura 1.8: Flujo de trabajo: del dato en crudo a la evaluación en el test oficial.

1.4.1. Problema 1: dependencia de la partición

Las métricas dependen de cómo se parta el dato. Para no depender de una única partición se usa validación cruzada. Ahora bien, en RAGTruth varias respuestas comparten el mismo documento fuente, y si un documento aparece a la vez en entrenamiento y validación el modelo memoriza en lugar de generalizar, inflando el resultado. Por eso se usa **GroupKFold agrupando por source**: cada documento cae entero en un solo fold (Figura 1.9). Un KFold aleatorio filtraría información entre folds. La cifra definitiva se reporta, además, sobre el *split oficial de test* de RAGTruth, que permanece intacto durante todo el desarrollo.

1.4.2. Problema 2: dependencia de los hiperparámetros

Los modelos con hiperparámetros pueden sobreajustarlos. Para elegirlos se usa *grid search*: se prueban combinaciones de hiperparámetros y se conserva la de mejor F1 en validación cruzada, sin tocar el test. Por ejemplo, optimizar tres hiperparámetros con tres valores cada uno con validación cruzada de cinco iteraciones exige $3 \cdot 3 \cdot 3 \cdot 5 = 135$ ajustes.

1.4.3. Problema 3: el desbalance y el punto de decisión

El tercer problema es propio de este trabajo. Como la prevalencia difiere por tarea (sección 1.1.5), un único umbral es subóptimo. La Figura 1.10 (izquierda) muestra cómo

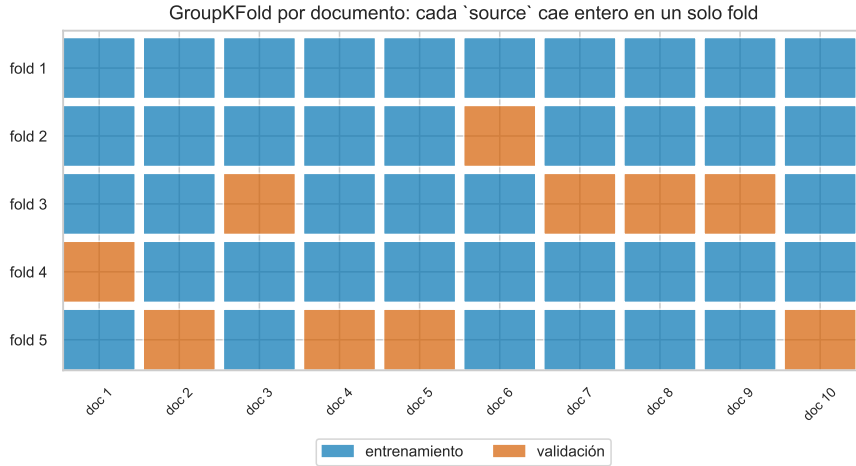
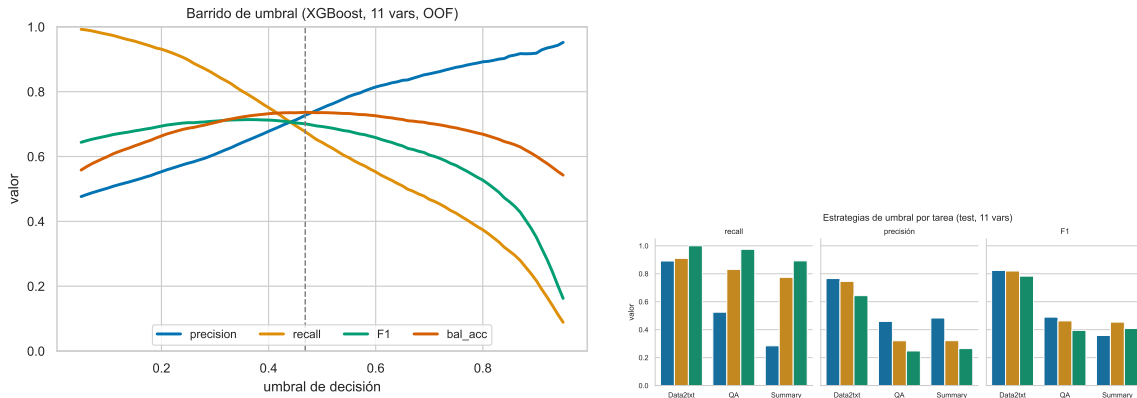


Figura 1.9: GroupKFold por documento: cada **source** cae entero en un fold, sin fuga entre entrenamiento y validación.

varían precisión, recall y F1 al mover el umbral. Se comparan tres criterios, todos con el umbral fijado en entrenamiento y aplicado al test: el índice de Youden global, el máximo de F1 por tarea, y el máximo de F_2 por tarea. El umbral *por tarea* (Figura 1.10, derecha) reconoce que cada tarea opera a una prevalencia distinta y recupera falsos negativos donde el umbral global los dejaba pasar, a costa de precisión. Como se verá en la sección 1.6.6, el umbral por tarea recorta los falsos negativos de 285 a 125.



(a) Métricas frente al umbral (OOF).

(b) Estrategias de umbral por tarea (test).

Figura 1.10: El umbral desliza el punto de operación entre precisión y recall; un umbral por tarea se adapta a cada prevalencia.

1.5. Marco de trabajo

El detector se implementa como una librería instalable, **ragcheck**, que descarga RAG-Truth, construye las características, entrena y evalúa los modelos y sirve la inferencia. Cada resultado de este capítulo es reproducible con semilla fija (42): mismo código y mismos datos producen siempre el mismo resultado. Los scripts de reentrenamiento y evaluación son atómicos, de modo que cada cifra tiene una fuente única y trazable.

1.6. Resultados por modelo

El propósito de esta sección es presentar los resultados de cada clasificador clásico sobre el conjunto de once variables, y elegir el mejor. Para cada modelo se indican sus hiperparámetros óptimos (por grid search con GroupKFold) y sus métricas en el test oficial, con la curva ROC. El umbral es el de Youden.

1.6.1. Regresión logística

La regresión logística [6] modela la probabilidad de alucinación con una función sigmoide de una combinación lineal de las características. El grid search elige regularización $C = 0,3$ con `class_weight=balanced`. Alcanza AUC 0,806 y F1 0,658 (Cuadro 1.5); su recall es el más alto (0,749) por el reponderado de clases, a costa de precisión.

1.6.2. K vecinos más cercanos

KNN [5] clasifica según las clases de los k vecinos más próximos, sobre las características estandarizadas. El grid search elige $k = 25$ vecinos ponderados por distancia. Da AUC 0,826 y F1 0,681, un rendimiento sólido.

1.6.3. Máquina de vectores de soporte

La SVM de núcleo RBF [4] busca una frontera de máximo margen en un espacio transformado, sobre las características estandarizadas. El grid search elige $C = 10$. Obtiene AUC 0,813 y el mejor F1 (0,690) y accuracy (0,784) de los cinco.

1.6.4. Bosque aleatorio

El bosque aleatorio [1] promedia muchos árboles de decisión sobre submuestras. El grid search elige profundidad máxima 20. Alcanza AUC 0,830 y F1 0,676.

1.6.5. Gradient boosting (XGBoost)

XGBoost [9, 3] construye árboles secuencialmente, corrigiendo el error del conjunto anterior. El grid search elige profundidad 6 y tasa de aprendizaje 0,05. Es el mejor en AUC (0,837) y en el promedio F1–AUC (Figura 1.11).

1.6.6. Comparación y modelo ganador

Para elegir el mejor modelo se usa, como criterio, el promedio entre F1 y AUC. El Cuadro 1.5 recoge las métricas de los cinco y la Figura 1.12 sus curvas. Las diferencias son pequeñas: los cinco superan el AUC 0,80 de la hipótesis. El mejor por promedio F1–AUC es **XGBoost** (0,763), que combina el mejor AUC con una buena calibración; será el modelo de referencia. Conviene notar que estas once variables igualan al conjunto completo de dieciocho (XGBoost con dieciocho da AUC 0,835), lo que confirma que la selección no pierde señal.

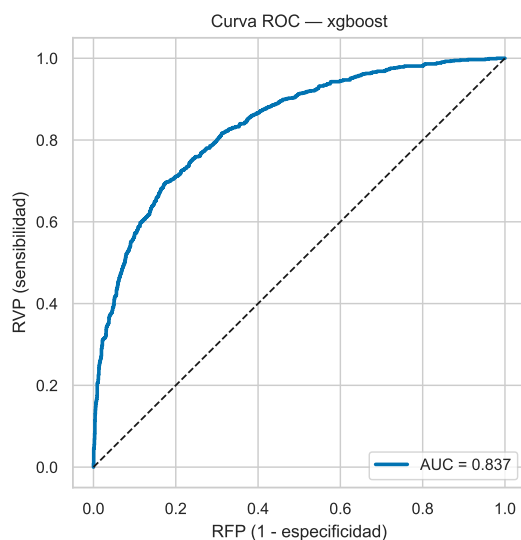
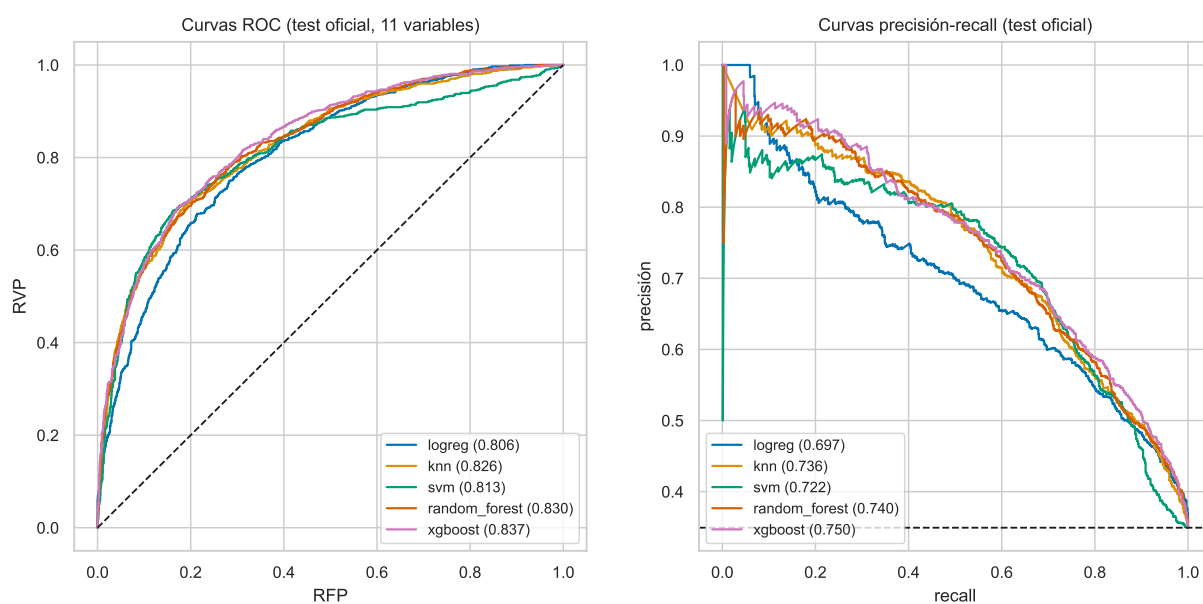


Figura 1.11: Curva ROC de XGBoost (test).

Modelo	AUC	F1	precisión	recall	accuracy	bal-acc	prom. F1–AUC
XGBoost	0,837	0,688	0,685	0,691	0,781	0,760	0,763
KNN	0,826	0,681	0,665	0,698	0,771	0,754	0,754
Bosque aleatorio	0,830	0,676	0,679	0,672	0,774	0,751	0,753
SVM	0,813	0,690	0,693	0,686	0,784	0,762	0,751
Reg. logística	0,806	0,658	0,587	0,749	0,729	0,733	0,732

Tabla 1.5: Comparación de los cinco modelos en el test oficial (once variables, umbral de Youden). Ganador por promedio F1–AUC: XGBoost.



(a) Curvas ROC.

(b) Curvas precisión-recall.

Figura 1.12: Discriminación de los cinco modelos sobre el test oficial (once variables).

1.6.7. El ganador por tarea y su calibración

El desglose por tarea del ganador (Cuadro 1.6) revela dónde está la dificultad. Data2txt y QA se separan bien (AUC 0,79 y 0,82); Summary se queda en 0,70. La paradoja de que QA tenga el mejor AUC y a la vez un F1 modesto se explica por la prevalencia: con solo un 18 % de positivos, un umbral global calla los positivos y el recall se hunde (0,525). Aquí actúa el umbral por tarea de la sección 1.4.3: al fijarlo por tarea, los falsos negativos totales bajan de 285 a 125 (Figura 1.13a), el recall de QA sube a 0,831 y el F1 de Summary de 0,358 a 0,454. La calibración del ganador es buena y mejora con regresión isotónica [15] (Figura 1.13b), lo que da probabilidades fiables para fijar el umbral. Las matrices de confusión por tarea (Figura 1.14) muestran visualmente el recorte de falsos negativos.

Tarea	prev.	AUC	F1	recall	accuracy	FN
Data2txt	0,643	0,786	0,824	0,891	0,754	63
QA	0,178	0,817	0,490	0,525	0,806	76
Summary	0,227	0,702	0,358	0,284	0,769	146
Global	0,349	0,837	0,685	0,698	0,776	285

Tabla 1.6: XGBoost por tarea (test oficial, umbral de Youden global). QA discrimina bien (AUC 0,82) pero calla positivos por su baja prevalencia.

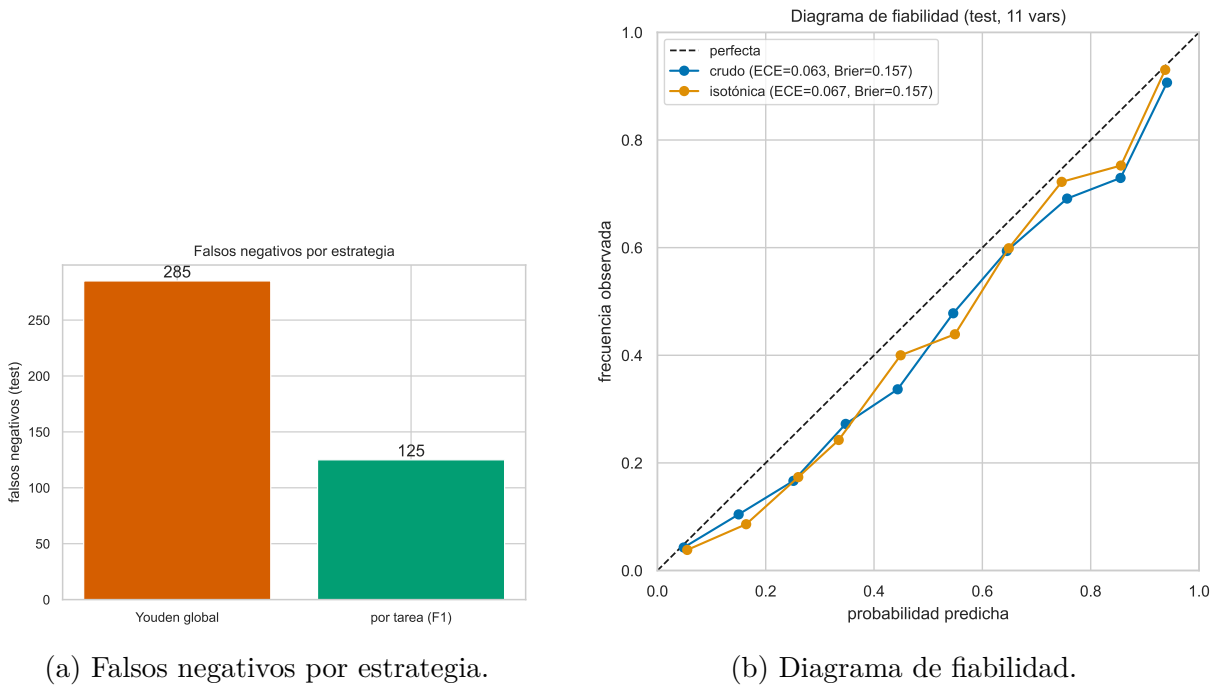


Figura 1.13: El umbral por tarea recorta el falso negativo; la calibración es buena y mejora con isotónica.

1.7. Comparación con el estado del arte

El estado del arte sobre RAGTruth se mide en F1 a nivel de respuesta y en accuracy, no en AUC. El Cuadro 1.7 y la Figura 1.15 sitúan nuestro detector por tarea.

Matrices de confusión por tarea (XGBoost, 11 vars, test)

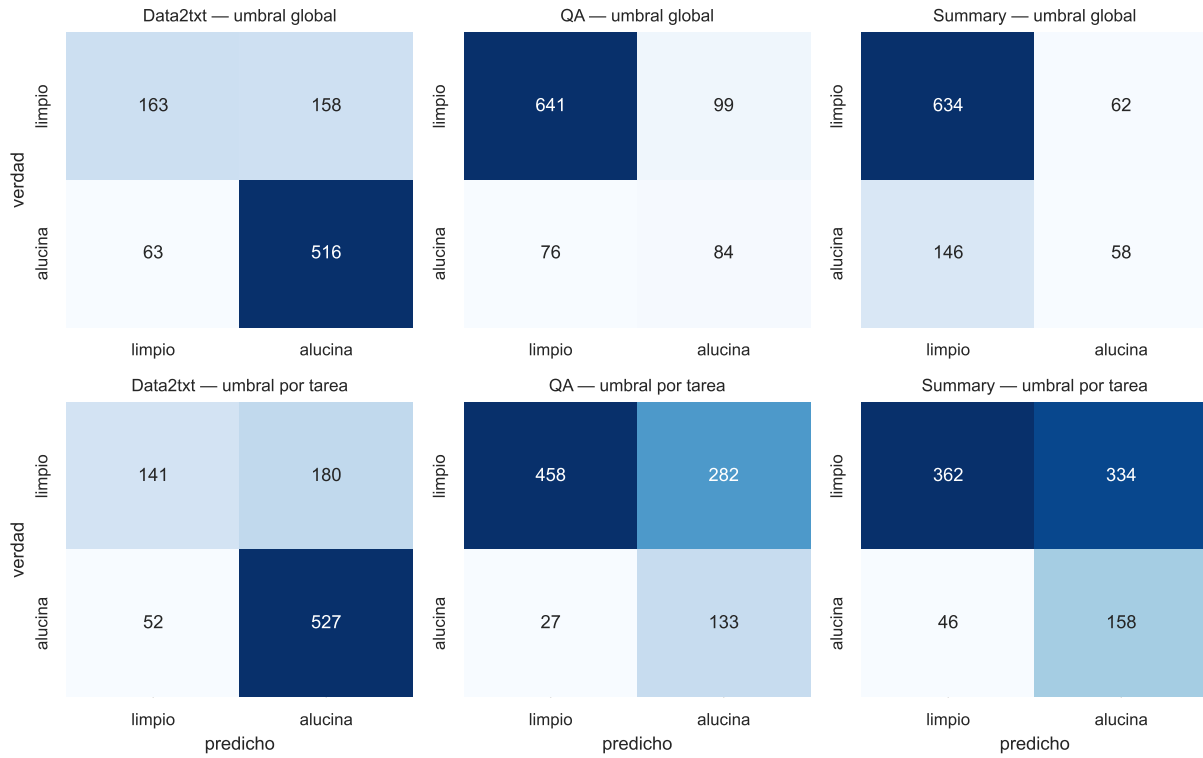


Figura 1.14: Matrices de confusión por tarea (XGBoost, test). Fila superior: umbral global; fila inferior: umbral por tarea. La detección de alucinaciones crece en QA y Summary.

Nota de comparabilidad. Todas las cifras están sobre RAGTruth y sobre la misma métrica (F1 a nivel de respuesta en el split oficial), pero los métodos de la literatura son *neuronales o basados en LLM* (encoders finetuneados o modelos de lenguaje como juez), mientras el nuestro es clásico sobre características léxicas. Además, la tabla de accuracy (Lynx) se reporta sobre el subconjunto de RAGTruth incluido en HaluBench, no sobre el split idéntico, y la literatura no publica su desglose por tarea. Se comparan, por tanto, cifras sobre el mismo problema y benchmark, no el mismo tipo de modelo.

Método (F1 nivel respuesta, %)	QA	Data2txt	Summary	Global
GPT-4-turbo (juez)	45,6	78,3	47,6	63,4
Luna (encoder ligero)	51,3	75,9	52,5	65,4
LettuceDetect-base (150M)	65,5	87,9	50,5	76,1
Fine-tuned Llama-2-13B	68,2	88,1	59,1	78,7
LettuceDetect-large (396M)	70,2	88,5	59,7	79,2
RAG-HAT (mejor)	74,8	91,6	67,6	83,9
Este trabajo (clásico)	46,3	82,0	45,4	68,5

Tabla 1.7: F1 por tarea sobre RAGTruth, recopilado por Kovács et al. [12] con RAG-HAT [19] como cota alta. Nuestro F1 por tarea usa umbral por tarea; el global (68,5) es el F1 *pooled* con umbral de Youden (el F1 macro por tarea es 55,7).

En accuracy, la referencia es Lynx [16]: GPT-3.5-turbo 50,7 %, Claude-3-Sonnet 79,1 %, Lynx-8B 80,0 %, Lynx-70B 80,2 % y GPT-4o 84,3 %; nuestra accuracy es 77,6 %. La lec-

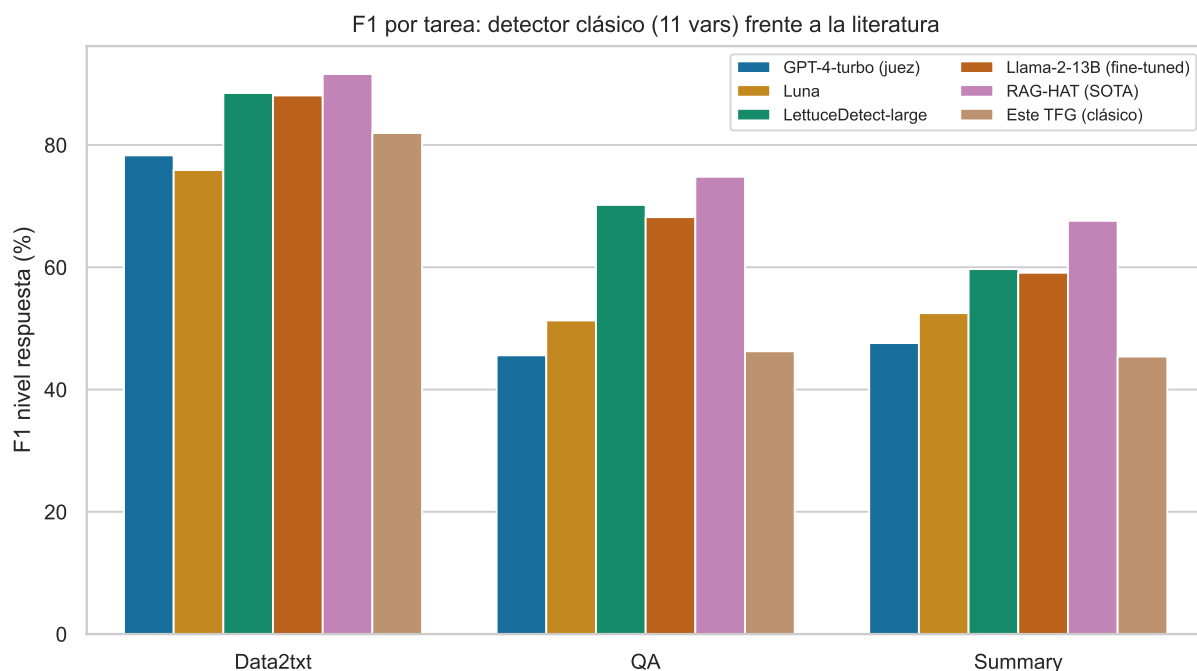


Figura 1.15: F1 por tarea: nuestro detector clásico frente a la literatura. La brecha se concentra en Summary, la tarea que nadie resuelve con solvencia.

tura honesta es doble. En F1 global superamos al juez GPT-4-turbo (63,4) y a Luna (65,4), y quedamos unos diez puntos por debajo de los modelos neuronales ajustados. El patrón por tarea es el mismo para todos: Data2txt fácil (88–92 %), QA y Summary mucho más bajos. Chocamos contra el mismo muro que modelos de miles de millones de parámetros, solo un poco antes, a coste cero y en CPU.

1.8. Exploración: vías descartadas

El resultado anterior es el destilado de una exploración amplia. Se recogen aquí, de forma condensada, las vías que no funcionaron, porque forman parte de la caracterización del problema.

Características que no aportaron. Se probaron y descartaron, todas medidas con el protocolo honesto: LSA (TF-IDF con descomposición en valores singulares truncada [7]), que aportó +0,001 de AUC; la novedad de n -gramas por frase, +0,001; y características estructurales de spaCy (arcos de dependencia, sujeto-verbo-objeto, enlace entidad-número), que dejaron Summary en 0,701 $\rightarrow$ 0,700.

Tratamiento del desbalance que no movió la frontera. Reequilibrar las clases no crea señal: el sobremuestreo, el submuestreo, SMOTE [2] y el aprendizaje sensible al coste dejan la frontera (AUC-PR por tarea) plana o peor (Figura 1.16a). La corrección de prior de Saerens [17] colapsa: su estimación por esperanza-maximización converge a prevalencias degeneradas (0 o 1), como muestra la Figura 1.16b. La única palanca honesta contra el falso negativo es el umbral por tarea de la sección 1.4.3.

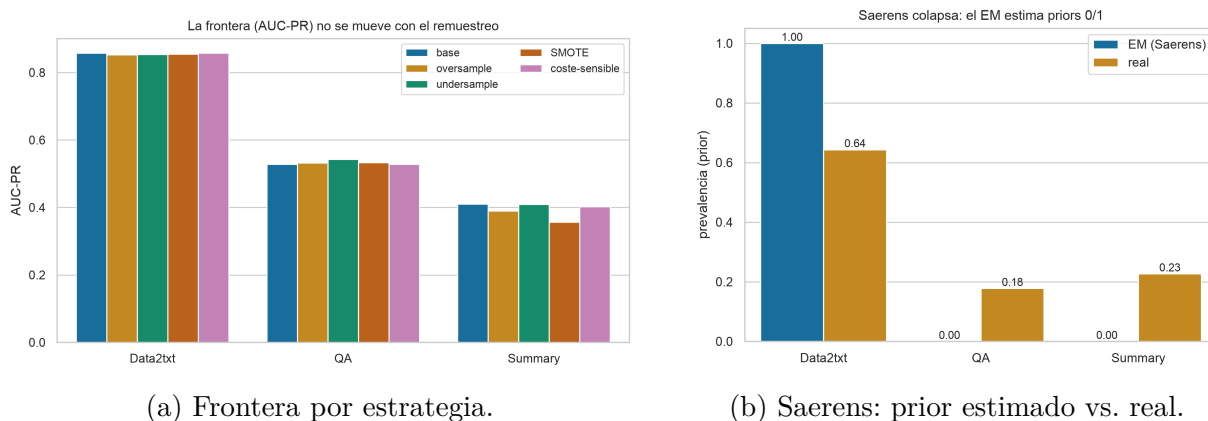


Figura 1.16: Reequilibrar clases no mueve la frontera; la corrección de prior de Saerens colapsa. Resultados negativos.

El techo de Summary. Summary es un techo estructural del enfoque léxico. Un resumen reutiliza el vocabulario del documento, así que el tramo alucinado comparte palabras con la fuente aunque el error sea evidente: su solape es 0,62 de mediana frente a 0,43 en Data2txt, y dentro de Summary los falsos negativos tienen más solape (0,641) que los aciertos (0,555) (Figura 1.17). Ninguna característica de superficie distingue el resumen fiel del alucinado.

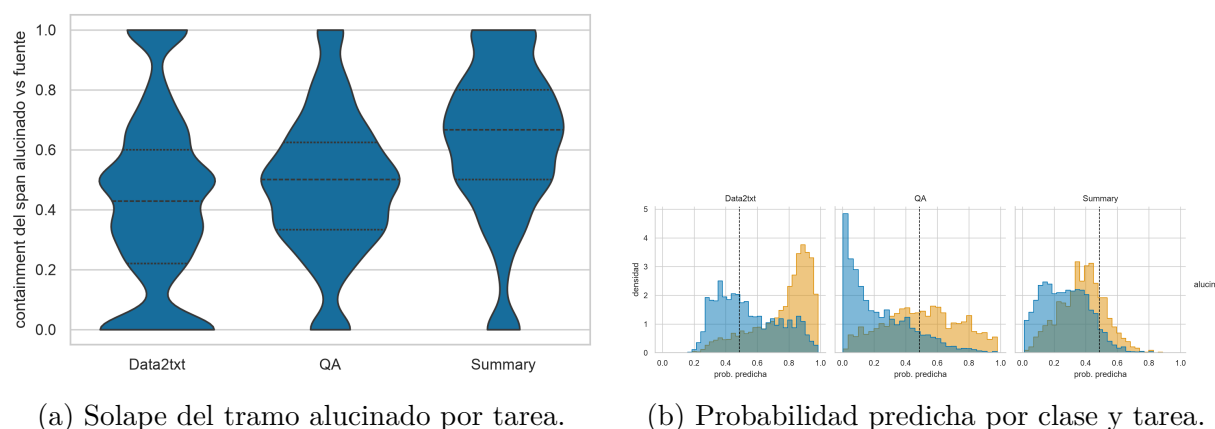


Figura 1.17: El cuello de botella de Summary es el camuflaje léxico, no el tipo de alucinación.

1.9. Conclusiones

Hipótesis. Se cumple. Los cinco clasificadores clásicos superan el AUC-ROC de 0,80 sobre el test oficial con evaluación honesta, y el mejor, XGBoost, alcanza 0,837 con solo once características léxicas seleccionadas por RFE.

Falsos negativos y desbalance. El déficit de detección no era de características ni de balance de clases, sino del punto de decisión bajo un desbalance heterogéneo. Un umbral por tarea recupera casi la mitad de los falsos negativos (de 285 a 125) y hace competitivo el F1 de Summary. La corrección de prior, el remuestreo y el aprendizaje sensible al coste se descartaron con evidencia.

Posicionamiento. El detector rinde por encima del juez GPT-4-turbo y de Luna, y a unos diez puntos de los detectores neuronales ajustados, a coste cero, en CPU, de forma interpretable y determinista. El techo de Summary queda caracterizado, no supuesto: es la frontera entre la minería léxica de superficie y la semántica profunda, y el estado del arte, con modelos mucho mayores, tropieza con el mismo muro.

Bibliografía

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1), 2001.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [4] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3), 1995.
- [5] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 1967.
- [6] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 1958.
- [7] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 1990.
- [8] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 2006.
- [9] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001.
- [10] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3), 2002.
- [11] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.
- [12] Ádám Kovács and Gábor Recski. LettuceDetect: A hallucination detection framework for RAG applications. *arXiv preprint arXiv:2502.17125*, 2025.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [14] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [15] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.
- [16] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, et al. Lynx: An open source hallucination evaluation model. *arXiv preprint arXiv:2407.08488*, 2024.
- [17] Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 2002.
- [18] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 1988.
- [19] Juntong Song, Xingguang Wang, Juno Zhu, et al. RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. In *Proceedings of EMNLP 2024 (Industry Track)*, 2024.